

Arrays:

1. Develop a Java program that declares and initializes an array of integers. Print the elements of the array in reverse order.

```
public class ReverseArray {  
  
    public static void main(String[] args) {  
  
        // 1. Declare and initialize the array  
  
        int[] numbers = { 10, 20, 30, 40, 50 };  
  
  
        // 2. Print the elements in reverse order  
  
        System.out.print("Array in reverse order: ");  
  
        for (int i = numbers.length - 1; i >= 0; i--) {  
  
            System.out.print(numbers[i] + " ");  
  
        }  
  
    }  
  
}
```

Output

Clear

```
Array in reverse order: 50 40 30 20 10
=== Code Execution Successful ===
```

2. Implement a Java program that finds the sum and average of elements in an array of floating-point numbers.

```
public class SumAndAverage {
    public static void main(String[] args) {
        // 1. Declare and initialize the array of floating-point numbers
        double[] values = { 3.5, 7.2, 1.8, 9.0, 4.6 };

        // 2. Compute the sum
        double sum = 0.0;
        for (double v : values) {
            sum += v;
        }

        // 3. Compute the average
        double average = sum / values.length;

        // 4. Print results
        System.out.printf("Sum of array elements: %.2f\n", sum);
    }
}
```

```
        System.out.printf("Average of array elements: %.2f\n", average);  
    }  
}
```

OutputClear

Sum of array elements: 26.10
Average of array elements: 5.22

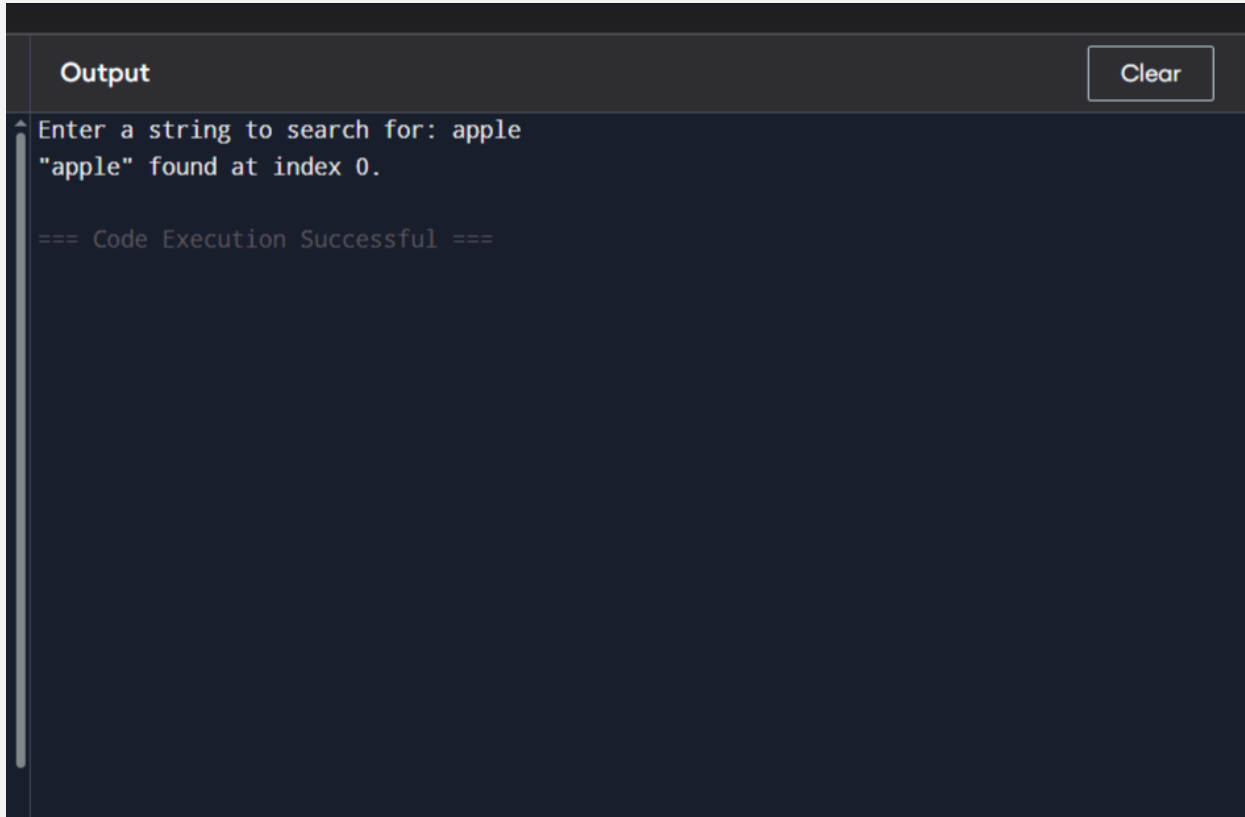
=== Code Execution Successful ===

3. Write a Java program that checks if a given element is present in an array of strings. If present, print its index; otherwise, print a message indicating its absence.

```
import java.util.Scanner;  
  
public class SearchStringInArray {  
    public static void main(String[] args) {  
        // 1. Declare and initialize the array of strings  
        String[] items = { "apple", "banana", "cherry", "date",  
"elderberry" };  
  
        // 2. Read the target string from the user  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Enter a string to search for: ");  
        String target = scanner.nextLine();  
        scanner.close();  
    }  
}
```

```
// 3. Search the array
int foundIndex = -1;
for (int i = 0; i < items.length; i++) {
    if (items[i].equals(target)) {
        foundIndex = i;
        break;
    }
}

// 4. Print the result
if (foundIndex != -1) {
    System.out.println "\"" + target + "\" found at index " +
foundIndex + ".");
} else {
    System.out.println "\"" + target + "\" is not present in the
array.");
}
}
```



The screenshot shows a dark-themed IDE window titled "Output". In the top right corner, there is a button labeled "Clear". The main area of the window displays the following text: "Enter a string to search for: apple", "apple" found at index 0., and "=== Code Execution Successful ===". A vertical scrollbar is visible on the left side of the output area.

4. Create a Java program that sorts an array of integers in ascending order using the bubble sort algorithm.

```
public class BubbleSort {  
  
    public static void main(String[] args) {  
  
        // 1. Declare and initialize the array of integers  
  
        int[] numbers = { 64, 34, 25, 12, 22, 11, 90 };  
  
        System.out.print("Original array: ");  
  
        printArray(numbers);  
  
        // 2. Sort the array using bubble sort
```

```
        bubbleSort(numbers);

        System.out.print("Sorted array:   ");

        printArray(numbers);

    }

    // Method to perform bubble sort on an integer array
    public static void bubbleSort(int[] arr) {

        int n = arr.length;

        // Traverse through all array elements
        for (int i = 0; i < n - 1; i++) {

            // Last i elements are already in place
            for (int j = 0; j < n - 1 - i; j++) {

                // Swap if the element found is greater than the next
element

                if (arr[j] > arr[j + 1]) {

                    int temp = arr[j];

                    arr[j] = arr[j + 1];

                    arr[j + 1] = temp;

                }

            }

        }

    }

}
```

```
// Helper method to print the contents of the array

public static void printArray(int[] arr) {

    for (int num : arr) {

        System.out.print(num + " ");

    }

    System.out.println();

}

}
```

Output

Clear

Original array: 64 34 25 12 22 11 90
Sorted array: 11 12 22 25 34 64 90

=== Code Execution Successful ===

Basics of Classes and Objects:

5. Define a class named BankAccount with attributes accountNumber, balance, and accountHolderName, accountHolderAddress.

```
// Question Number 5 //
```

```
public class BankAccount {  
  
    // Attributes  
  
    private String accountNumber;  
  
    private double balance;  
  
    private String accountHolderName;  
  
    private String accountHolderAddress;  
  
  
    // No-argument constructor  
  
    public BankAccount() {  
  
        // You could set default values here if you like  
  
    }  
  
  
    // Full-argument constructor  
  
    public BankAccount(String accountNumber,  
  
                        double balance,
```



```
        String accountHolderName,  
        String accountHolderAddress) {  
  
    this.accountNumber = accountNumber;  
  
    this.balance = balance;  
  
    this.accountHolderName = accountHolderName;  
  
    this.accountHolderAddress = accountHolderAddress;  
  
}  
  
// Getters  
  
public String getAccountNumber() {  
  
    return accountNumber;  
  
}  
  
public double getBalance() {  
  
    return balance;  
  
}  
  
public String getAccountHolderName() {  
  
    return accountHolderName;  
  
}  
  
public String getAccountHolderAddress() {  
  
    return accountHolderAddress;  
  
}
```

```
}

// Setters

public void setAccountNumber(String accountNumber) {

    this.accountNumber = accountNumber;

}

public void setBalance(double balance) {

    this.balance = balance;

}

public void setAccountHolderName(String accountHolderName) {

    this.accountHolderName = accountHolderName;

}

public void setAccountHolderAddress(String accountHolderAddress) {

    this.accountHolderAddress = accountHolderAddress;

}

// toString override for easy printing

@Override

public String toString() {

    return "BankAccount{" +
```

```
        "accountNumber='" + accountNumber + '\'' +  
        ", balance=" + balance +  
        ", accountHolderName='" + accountHolderName + '\'' +  
        ", accountHolderAddress='" + accountHolderAddress + '\'' +  
        '}' ;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // Create and initialize a BankAccount object using the  
full-argument constructor  
        BankAccount account = new BankAccount(  
            "ACC123456789",           // accountNumber  
            2500.50,                  // balance  
            "Jane Doe",              // accountHolderName  
            "742 Evergreen Terrace"  // accountHolderAddress  
        );  
  
        // Optionally, print out the account details  
        System.out.println(account);  
    }  
}
```

```
// Or access individual fields via getters

        System.out.println("Account    Number:    " +
account.getAccountNumber());

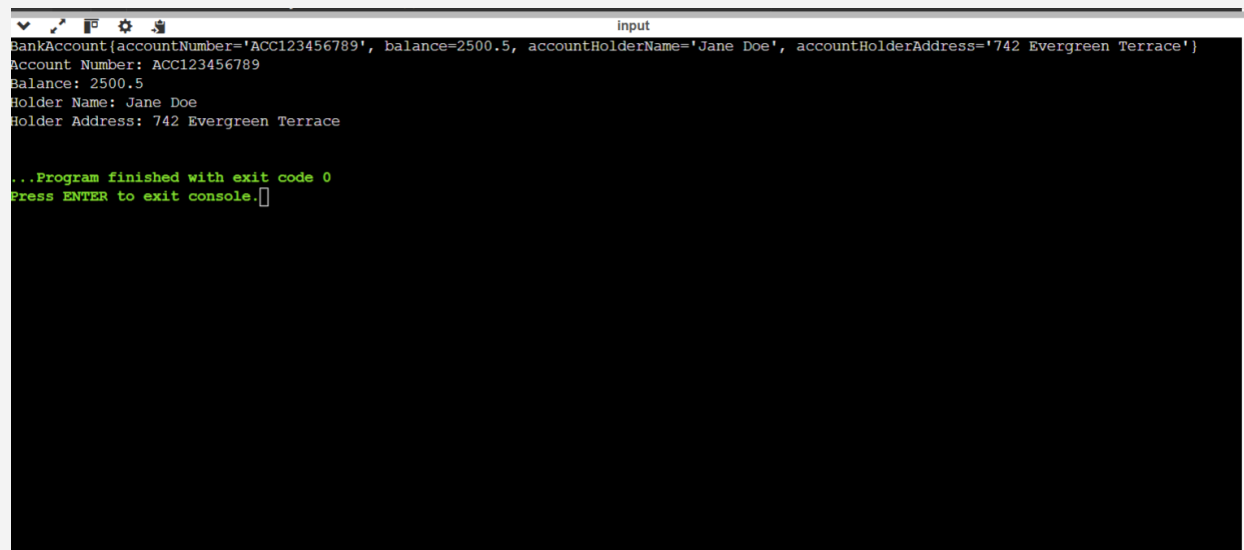
        System.out.println("Balance: " + account.getBalance());

        System.out.println("Holder    Name:    " +
account.getAccountHolderName());

        System.out.println("Holder    Address:    " +
account.getAccountHolderAddress());

    }

}
```



```
BankAccount{accountNumber='ACC123456789', balance=2500.5, accountHolderName='Jane Doe', accountHolderAddress='742 Evergreen Terrace'}
Account Number: ACC123456789
Balance: 2500.5
Holder Name: Jane Doe
Holder Address: 742 Evergreen Terrace

...Program finished with exit code 0
Press ENTER to exit console.
```

6. Create an object of this class and initialize its attributes.

```
// Question Number 6 //
```

```
public class BankAccount2 {  
  
    // Attributes  
  
    private String accountNumber;  
  
    private double balance;  
  
    private String accountHolderName;  
  
    private String accountHolderAddress;  
  
  
    // No-argument constructor  
  
    public BankAccount2() {  
  
        // Default constructor  
  
    }  
  
  
    // Full-argument constructor  
  
    public BankAccount2(String accountNumber,  
  
                        double balance,  
  
                        String accountHolderName,  
  
                        String accountHolderAddress) {  
  
        this.accountNumber = accountNumber;  
  
        this.balance = balance;  
  
        this.accountHolderName = accountHolderName;  
  
        this.accountHolderAddress = accountHolderAddress;  
  
    }  
}
```

```
// Getters

public String getAccountNumber() {

    return accountNumber;

}

public double getBalance() {

    return balance;

}

public String getAccountHolderName() {

    return accountHolderName;

}

public String getAccountHolderAddress() {

    return accountHolderAddress;

}

// Setters

public void setAccountNumber(String accountNumber) {

    this.accountNumber = accountNumber;

}
```

```
public void setBalance(double balance) {

    this.balance = balance;

}

public void setAccountHolderName(String accountHolderName) {

    this.accountHolderName = accountHolderName;

}

public void setAccountHolderAddress(String accountHolderAddress) {

    this.accountHolderAddress = accountHolderAddress;

}

// toString override for easy printing

@Override

public String toString() {

    return "BankAccount{" +

        "accountNumber='" + accountNumber + '\'' +

        ", balance=" + balance +

        ", accountHolderName='" + accountHolderName + '\'' +

        ", accountHolderAddress='" + accountHolderAddress + '\'' +

        '}';

}
```

```
// Method to deposit money into the account

public void depositMoney(double amount) {

    if (amount > 0) {

        balance += amount;

        System.out.printf("Deposited %.2f. Current balance: %.2f\n",
amount, balance);

    } else {

        System.out.println("Deposit amount must be positive.");

    }

}

// Method to withdraw money from the account

public void withdrawMoney(double amount) {

    if (amount > 0) {

        if (balance >= amount) {

            balance -= amount;

            System.out.printf("Withdrew %.2f. Current balance:
%.2f\n", amount, balance);

        } else {

            System.out.println("Insufficient balance to withdraw " +
amount);

        }

    } else {

        System.out.println("Withdrawal amount must be positive.");

    }

}
```



```
}  
  
}  
  
}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop3> & 'C:\Program Files\Eclipse  
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'  
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Yajju  
Shrestha\AppData\Roaming\Code\User\workspaceStorage\3491f3445  
524c75c422c79d5e10d4c58\redhat.java\jdt_ws\Workshop3_34f5a26c\  
bin' 'Main'
```

```
New          account          created:  
BankAccount{accountNumber='ACC100200300', balance=1500.75,  
accountHolderName='John Smith', accountHolderAddress='221B  
Baker Street'}
```

```
PS E:\jaca\Workshop3>
```

Methods:

7. Create a method, **depositMoney()** in the BankAccount class to deposit money. Implement another method, **withdrawMoney()** to withdraw money. (The current balance should also be printed).

```
// Question Number 7 //
```

```
public class BankAccount3 {  
  
    // Attributes  
  
    private String accountNumber;  
  
    private double balance;  
  
    private String accountHolderName;  
  
    private String accountHolderAddress;  
  
  
    // Parameterized constructor to initialize all attributes  
  
    public BankAccount3(String accountNumber,  
                        double balance,  
                        String accountHolderName,  
                        String accountHolderAddress) {  
  
        this.accountNumber = accountNumber;  
  
        this.balance = balance;  
  
        this.accountHolderName = accountHolderName;  
  
        this.accountHolderAddress = accountHolderAddress;  
  
    }  
}
```

```
// Getter methods

public String getAccountNumber() { return accountNumber; }

public double getBalance() { return balance; }

public String getAccountHolderName() { return accountHolderName; }

public String getAccountHolderAddress() { return accountHolderAddress; }
}

// Method to deposit money into the account

public void depositMoney(double amount) {

    if (amount > 0) {

        balance += amount;

        System.out.printf("Deposited %.2f. Current balance: %.2f\n",
amount, balance);

    } else {

        System.out.println("Deposit amount must be positive.");

    }

}

// Method to withdraw money from the account

public void withdrawMoney(double amount) {

    if (amount > 0) {

        if (balance >= amount) {

            balance -= amount;

        }

    }

}
```

```
        System.out.printf("Withdrew %.2f. Current balance: %.2f\n", amount, balance);

    } else {

        System.out.println("Insufficient balance to withdraw " + amount);

    }

    } else {

        System.out.println("Withdrawal amount must be positive.");

    }

}

@Override

public String toString() {

    return String.format(

        "BankAccount{accountNumber='%s', balance=%.2f, accountHolderName='%s', accountHolderAddress='%s'}",

        accountNumber, balance, accountHolderName, accountHolderAddress

    );

}

}

// Main.java (placed in the same file as package-private class to avoid public conflict)

class Main {
```

```
public static void main(String[] args) {  
  
    // Create a BankAccount object using the parameterized constructor  
  
    BankAccount myAccount = new BankAccount(  
  
        "ACC100200300",    // accountNumber  
  
        1500.75,           // initial balance  
  
        "John Smith",      // account holder name  
  
        "221B Baker Street" // account holder address  
  
    );  
  
    // Print initial account details  
  
    System.out.println("New account created: " + myAccount);  
  
}  
}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop3> & 'C:\Program Files\Eclipse  
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'  
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Yajju  
Shrestha\AppData\Roaming\Code\User\workspaceStorage\3491f3445  
524c75c422c79d5e10d4c58\redhat.java\jdt_ws\Workshop3_34f5a26c\  
bin' 'Main'
```

New account created:
BankAccount{accountNumber='ACC100200300', balance=1500.75,
accountHolderName='John Smith', accountHolderAddress='221B
Baker Street'}

PS E:\jaca\Workshop3>

Constructors:

8. Implement a parameterized constructor for the BankAccount class that initializes the account attributes. Create an object using this constructor.

```
// Question Number 8 //
```

```
public class BankAccount4 {  
  
    // Attributes  
  
    private String accountNumber;  
  
    private double balance;  
  
    private String accountHolderName;  
  
    private String accountHolderAddress;  
  
  
    // No-argument constructor  
  
    public BankAccount4() {
```

```
        System.out.println("User created!");
    }

    // Parameterized constructor to initialize all attributes
    public BankAccount4(String accountNumber,
                        double balance,
                        String accountHolderName,
                        String accountHolderAddress) {

        this.accountNumber = accountNumber;

        this.balance = balance;

        this.accountHolderName = accountHolderName;

        this.accountHolderAddress = accountHolderAddress;
    }

    // Getter methods

    public String getAccountNumber() { return accountNumber; }

    public double getBalance() { return balance; }

    public String getAccountHolderName() { return accountHolderName; }

    public String getAccountHolderAddress() { return accountHolderAddress;
}

    // Method to deposit money into the account

    public void depositMoney(double amount) {

        if (amount > 0) {
```

```
        balance += amount;

        System.out.printf("Deposited %.2f. Current balance: %.2f\n",
amount, balance);

    } else {

        System.out.println("Deposit amount must be positive.");

    }

}

// Method to withdraw money from the account

public void withdrawMoney(double amount) {

    if (amount > 0) {

        if (balance >= amount) {

            balance -= amount;

            System.out.printf("Withdrew %.2f. Current balance:
%.2f\n", amount, balance);

        } else {

            System.out.println("Insufficient balance to withdraw " +
amount);

        }

    } else {

        System.out.println("Withdrawal amount must be positive.");

    }

}
```



```
@Override

    public String toString() {

        return String.format(

            "BankAccount{accountNumber='%s',    balance=%.2f,
accountHolderName='%s', accountHolderAddress='%s'}",

            accountNumber,    balance,    accountHolderName,
accountHolderAddress

        );

    }

}

// Main.java (placed in the same file to avoid public conflict)

class Main {

    public static void main(String[] args) {

        // Create a BankAccount object using the no-argument constructor

        BankAccount newUserAccount = new BankAccount(); // prints "User
created!"

        // Use the newUserAccount to avoid unused-variable warning

        System.out.println("Default account state: " + newUserAccount);

        // Create a BankAccount object using the parameterized constructor

        BankAccount myAccount = new BankAccount(

            "ACC100200300",    // accountNumber

            1500.75,    // initial balance
```

```
        "John Smith",           // account holder name

        "221B Baker Street" // account holder address

    );

    // Print initial account details

    System.out.println("New account created: " + myAccount);

}

}
```

```
PS E:\jaca\Workshop3> & 'C:\Program Files\Eclipse
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Yajju
Shrestha\AppData\Roaming\Code\User\workspaceStorage\3491f3445
524c75c422c79d5e10d4c58\redhat.java\jdt_ws\Workshop3_34f5a26c\
bin' 'Main'
```

```
New                account                created:
BankAccount{accountNumber='ACC100200300',  balance=1500.75,
accountHolderName='John  Smith',  accountHolderAddress='221B
Baker Street'}
```

```
PS E:\jaca\Workshop3>
```

9. Implement a no-argument constructor that prints out **“User created!”** as soon as the instance of the user is created.

```
// Question Number 9 //

public class BankAccount5 {

    // Attributes

    private String accountNumber;

    private double balance;

    private String accountHolderName;

    private String accountHolderAddress;

    // No-argument constructor

    public BankAccount5() {

        System.out.println("User created!");

    }

    // Parameterized constructor to initialize all attributes

    public BankAccount5(String accountNumber,

                        double balance,

                        String accountHolderName,

                        String accountHolderAddress) {

        this.accountNumber = accountNumber;

        this.balance = balance;

    }

}
```

```
        this.accountHolderName = accountHolderName;

        this.accountHolderAddress = accountHolderAddress;

    }

    // Getter methods

    public String getAccountNumber() { return accountNumber; }

    public double getBalance() { return balance; }

    public String getAccountHolderName() { return accountHolderName; }

    public String getAccountHolderAddress() { return accountHolderAddress; }
}

// Method to deposit money into the account

public void depositMoney(double amount) {

    if (amount > 0) {

        balance += amount;

        System.out.printf("Deposited %.2f. Current balance: %.2f\n",
amount, balance);

    } else {

        System.out.println("Deposit amount must be positive.");

    }

}

// Method to withdraw money from the account

public void withdrawMoney(double amount) {
```

```
        if (amount > 0) {

            if (balance >= amount) {

                balance -= amount;

                System.out.printf("Withdrew %.2f. Current balance: %.2f\n", amount, balance);

            } else {

                System.out.println("Insufficient balance to withdraw " + amount);

            }

        } else {

            System.out.println("Withdrawal amount must be positive.");

        }

    }

}

@Override

public String toString() {

    return String.format(

        "BankAccount5{accountNumber='%s', balance=%.2f, accountHolderName='%s', accountHolderAddress='%s'}",

        accountNumber, balance, accountHolderName, accountHolderAddress

    );

}

}
```

```
// --- Main.java ---

class Main {

    public static void main(String[] args) {

        // Instantiate with no-argument constructor (prints "User
        created!")

        BankAccount5 defaultAccount = new BankAccount5();

        // Print default account details to use the variable

        System.out.println("Default account: " + defaultAccount);

        // Instantiate with parameterized constructor

        BankAccount5 myAccount = new BankAccount5(

            "ACC100200300",    // accountNumber

            1500.75,          // initial balance

            "John Smith",     // account holder name

            "221B Baker Street" // account holder address

        );

        // Print account details

        System.out.println("New account created: " + myAccount);

    }

}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop3> & 'C:\Program Files\Eclipse  
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'  
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Yajju  
Shrestha\AppData\Roaming\Code\User\workspaceStorage\3491f3445  
524c75c422c79d5e10d4c58\redhat.java\jdt_ws\Workshop3_34f5a26c\  
bin' 'Main'
```

```
New          account          created:  
BankAccount{accountNumber='ACC100200300', balance=1500.75,  
accountHolderName='John Smith', accountHolderAddress='221B  
Baker Street'}
```

10. Create a class `Lamp` with attributes **isOn** to store boolean value. Also create a method **turnOn()** to turn on the light, and **turnOff()** to turn off the light and print the on status of the light.

```
// Question Number 10  
  
public class Lamp {  
    private boolean isOn;
```

```
// No-argument constructor: lamp starts off

public Lamp() {

    this.isOn = false;

    System.out.println("Lamp created! Current state: " + (isOn ? "On"
: "Off"));

}

// Turn on the lamp and print status

public void turnOn() {

    isOn = true;

    System.out.println("Lamp is now On");

}

// Turn off the lamp and print status

public void turnOff() {

    isOn = false;

    System.out.println("Lamp is now Off");

}

// Getter for isOn

public boolean isOn() {

    return isOn;

}
```



```
}
```

```
// Question Number 10
```

```
public class MainLamp {  
  
    public static void main(String[] args) {  
  
        // Create Lamp instance (prints creation state)  
  
        Lamp lamp = new Lamp();  
  
  
        // Toggle lamp state  
  
        lamp.turnOn();    // prints "Lamp is now On"  
  
        lamp.turnOff();   // prints "Lamp is now Off"  
  
  
        // Optionally check state programmatically  
  
        System.out.println("Final lamp state: " + (lamp.isOn() ? "On" :  
"Off"));  
  
    }  
  
}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop3> & 'C:\Program Files\Eclipse  
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'  
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Yajju  
Shrestha\AppData\Roaming\Code\User\workspaceStorage\3491f3445  
524c75c422c79d5e10d4c58\redhat.java\jdt_ws\Workshop3_34f5a26c\  
bin' 'MainLamp'
```

Lamp created! Current state: Off

Lamp is now On

Lamp is now Off

Final lamp state: Off

Constructor Overloading:

11. Create a class named ,"**Box**" with attributes **width**, **height**, and **depth**. Create multiple constructors for handling following object declarations. Also declare a method **getVolume()** that prints the volume of the declared:
 - a. For a cube, declare a constructor to take length only.

- b. For a cuboid, declare a constructor to take length, breadth, and height.
- c. For no parameter, declare a no-argument constructor that sets **length = 10**, **breadth = 8**, and **height = 12**.

```
// Question Number 11

public class Box {

    // Attributes for Box dimensions

    private double length;

    private double breadth;

    private double height;

    // No-argument constructor setting default values

    public Box() {

        this.length = 10;

        this.breadth = 8;

        this.height = 12;

    }

    // Constructor for Cube (same length, breadth, height)

    public Box(double length) {

        this.length = length;
```

```
        this.breadth = length;    // For a cube, all sides are the
same

        this.height = length;

    }

    // Constructor for Cuboid (length, breadth, height)

    public Box(double length, double breadth, double height) {

        this.length = length;

        this.breadth = breadth;

        this.height = height;

    }

    // Method to calculate and print the volume

    public void getVolume() {

        double volume = length * breadth * height;

        System.out.println("The volume of the box is: " + volume);

    }

    // Main method to test the constructors

    public static void main(String[] args) {

        // Create a cube with length 5

        Box cube = new Box(5);

        cube.getVolume();    // Should print the volume of the cube
    }
}
```

```
// Create a cuboid with different dimensions

Box cuboid = new Box(4, 6, 10);

    cuboid.getVolume(); // Should print the volume of the
cuboid

// Create a box with default dimensions

Box defaultBox = new Box();

    defaultBox.getVolume(); // Should print the volume of the
default box

}

}
```

Output

Clear

```
The volume of the box is: 125.0
The volume of the box is: 240.0
The volume of the box is: 960.0
```

```
=== Code Execution Successful ===
```

Encapsulation:

12. Create a class Address with private attributes street, city, and zipCode. Use encapsulation and provide getter and setter methods.

```
public class Address {  
  
    // Private attributes  
  
    private String street;  
  
    private String city;  
  
    private String zipCode;  
  
    // Constructor to initialize Address object  
  
    public Address(String street, String city, String zipCode) {  
  
        this.street = street;  
  
        this.city = city;  
  
        this.zipCode = zipCode;  
  
    }  
  
    // Getter for street  
  
    public String getStreet() {  
  
        return street;  
  
    }  
  
}
```

```
}

// Setter for street

public void setStreet(String street) {

    this.street = street;

}

// Getter for city

public String getCity() {

    return city;

}

// Setter for city

public void setCity(String city) {

    this.city = city;

}

// Getter for zipCode

public String getZipCode() {

    return zipCode;

}

// Setter for zipCode
```

```
public void setZipCode(String zipCode) {

    this.zipCode = zipCode;

}

// Method to display the address in a formatted way

public void displayAddress() {

    System.out.println("Street: " + street);

    System.out.println("City: " + city);

    System.out.println("Zip Code: " + zipCode);

}

// Main method to test the Address class

public static void main(String[] args) {

    // Create an Address object

    Address address = new Address("123 Elm Street", "Springfield",
"98765");

    // Display the address using displayAddress method

    address.displayAddress();

    // Modify the address using setter methods

    address.setStreet("456 Oak Avenue");

    address.setCity("Greenwood");

    address.setZipCode("12345");
```



```
        // Display the updated address

        System.out.println("\nUpdated Address:");

        address.displayAddress();
    }
}
```

Output Clear

```
^
Street: 123 Elm Street
City: Springfield
Zip Code: 98765

Updated Address:
Street: 456 Oak Avenue
City: Greenwood
Zip Code: 12345

=== Code Execution Successful ===
```

Combining Concepts:

13. Create a class Customer with private attributes customerId, name, and a BankAccount attribute. Implement a parameterized constructor and encapsulate the attributes. Provide getter

method. Instantiate multiple Customer objects with different values and demonstrate the use of getters and setters.

```
//Question: 13

public class BankAccountNew {

    private String accountNumber;

    private double balance;

    // Constructor to initialize BankAccount

    public BankAccountNew(String accountNumber, double balance) {

        this.accountNumber = accountNumber;

        this.balance = balance;

    }

    // Getter for accountNumber

    public String getAccountNumber() {

        return accountNumber;

    }

    // Setter for accountNumber

    public void setAccountNumber(String accountNumber) {
```

```
        this.accountNumber = accountNumber;

    }

    // Getter for balance

    public double getBalance() {

        return balance;

    }

    // Setter for balance

    public void setBalance(double balance) {

        this.balance = balance;

    }

    // Method to display account details

    public void displayAccountDetails() {

        System.out.println("Account Number: " + accountNumber);

        System.out.println("Balance: $" + balance);

    }

}
```

```
//Question: 13
```

```
public class Customer {

    private int customerId;

    private String name;

    private BankAccountNew bankAccount; // Using BankAccountNew instead of
BankAccount

    // Parameterized constructor

    public Customer(int customerId, String name, BankAccountNew
bankAccount) {

        this.customerId = customerId;

        this.name = name;

        this.bankAccount = bankAccount;

    }

    // Getter for customerId

    public int getCustomerId() {

        return customerId;

    }

    // Setter for customerId

    public void setCustomerId(int customerId) {

        this.customerId = customerId;

    }

}
```

```
// Getter for name

public String getName() {

    return name;

}

// Setter for name

public void setName(String name) {

    this.name = name;

}

// Getter for bankAccount

public BankAccountNew getBankAccount() {

    return bankAccount;

}

// Setter for bankAccount

public void setBankAccount(BankAccountNew bankAccount) {

    this.bankAccount = bankAccount;

}

// Method to display customer details

public void displayCustomerDetails() {
```

```
        System.out.println("Customer ID: " + customerId);

        System.out.println("Customer Name: " + name);

        bankAccount.displayAccountDetails(); // Display associated bank
account details
    }

    // Main method to test the classes

    public static void main(String[] args) {

        // Create two BankAccountNew objects

        BankAccountNew account1 = new BankAccountNew("ACC12345", 1000.00);

        BankAccountNew account2 = new BankAccountNew("ACC67890", 2500.00);

        // Create two customers with their associated bank accounts

        Customer customer1 = new Customer(1, "Alice Johnson", account1);

        Customer customer2 = new Customer(2, "Bob Smith", account2);

        // Display customer details

        System.out.println("Customer 1 Details:");

        customer1.displayCustomerDetails();

        System.out.println();

        System.out.println("Customer 2 Details:");

        customer2.displayCustomerDetails();

    }
```

```
}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop3> & 'C:\Program Files\Eclipse  
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'  
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Yajju  
Shrestha\AppData\Roaming\Code\User\workspaceStorage\3491f3445  
524c75c422c79d5e10d4c58\redhat.java\jdt_ws\Workshop3_34f5a26c\  
bin' 'Customer'
```

Customer 1 Details:

Customer ID: 1

Customer Name: Alice Johnson

Account Number: ACC12345

Balance: \$1000.0

Customer 2 Details:

Customer ID: 2

Customer Name: Bob Smith

Account Number: ACC67890

Balance: \$2500.0

PS E:\jaca\Workshop3>